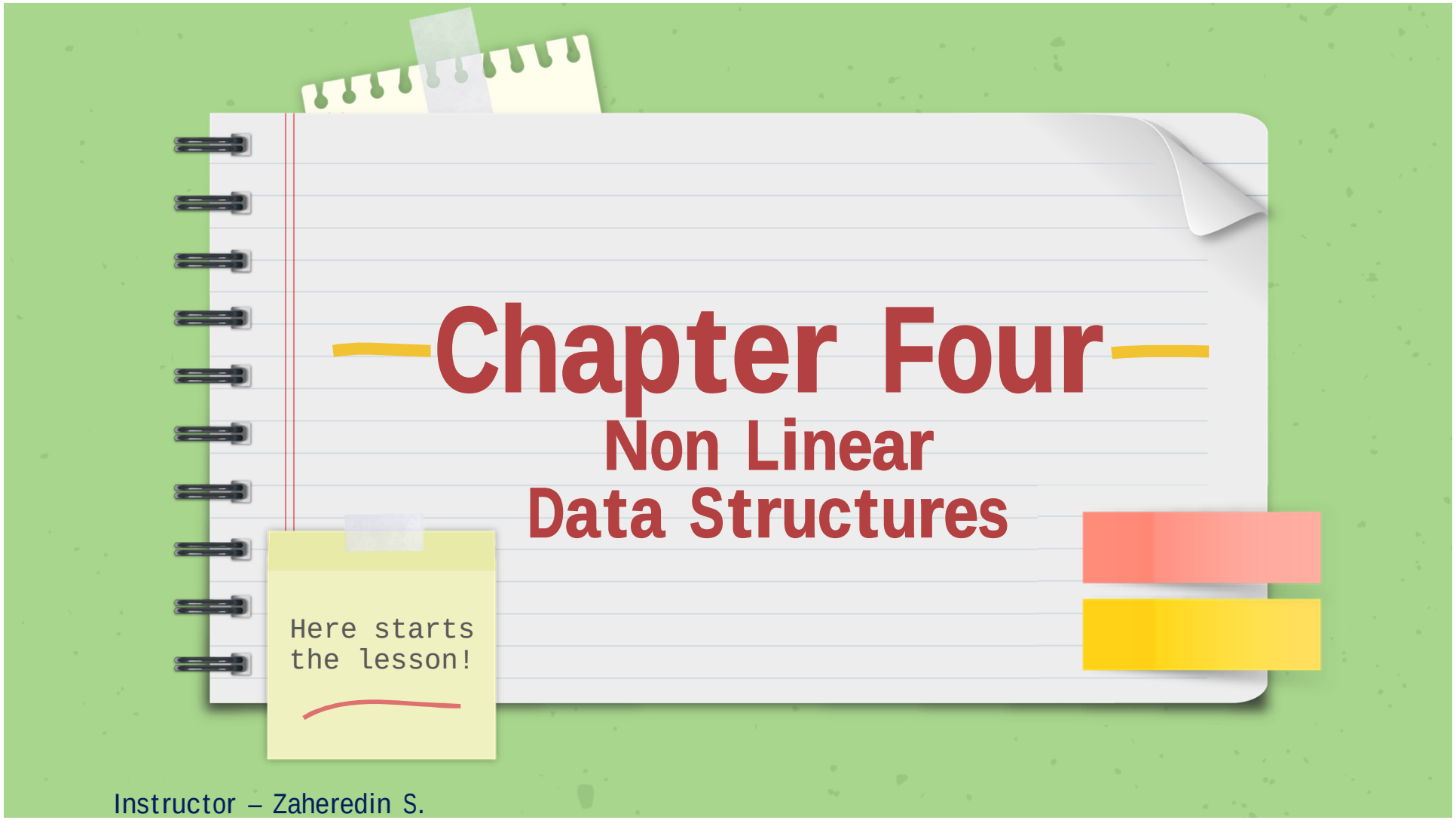


Instructor – Zaheredin S.

Chapter Four Contents!



- 4.1 Trees

Chapter Four Contents!



- 4.2 Graphs

4.1 - Tree

- **Trees** are a multilevel data structure containing the set of one or more data nodes where one node is designated as the root of the tree while the remaining nodes are called as the children of the root.
- The nodes other than the root node are partitioned into the non empty sets where each one of them is to be called sub-tree.
- Nodes of a tree either maintain a parent-child relationship between them or they are sister nodes.

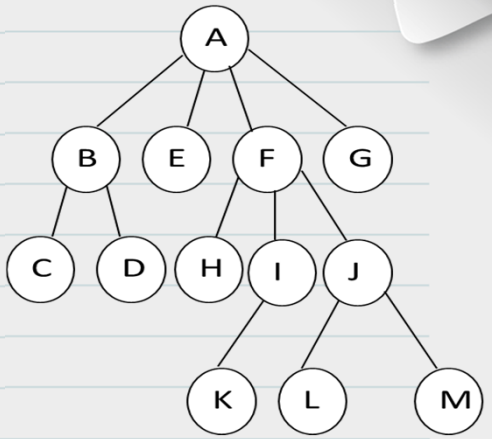
Cont...

- In a parent-child relationship, the bottom most nodes in the hierarchy are called leaf node while the topmost node is called root node.
- Its structure is similar to the real tree, so it is named a Tree. In this structure, the root is at the top, and its branches are moving in a downward direction.
- Therefore, we can say that the Tree data structure is an efficient way of storing the data in a hierarchical way.

Cont...

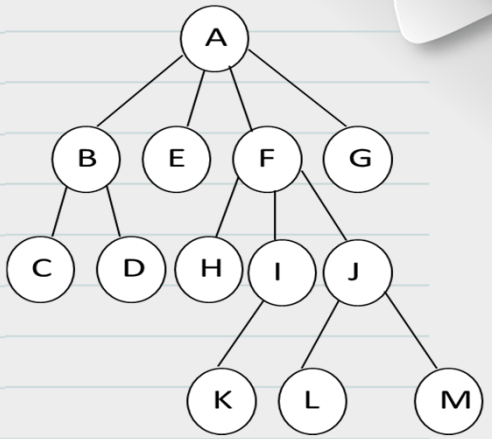
Basic Tree Terminology

- **Root**: a node with out a parent. → A
- **Internal node**: a node with at least one child. →A, B, F, I, J
- **External (leaf) node**: a node without a child.→ C, D, E, H, K, L, M, G
- **Ancestors of a node**: parent, grandparent, grand-grandparent, etc. of a node. Ancestors of K→ A, F, I



Cont...

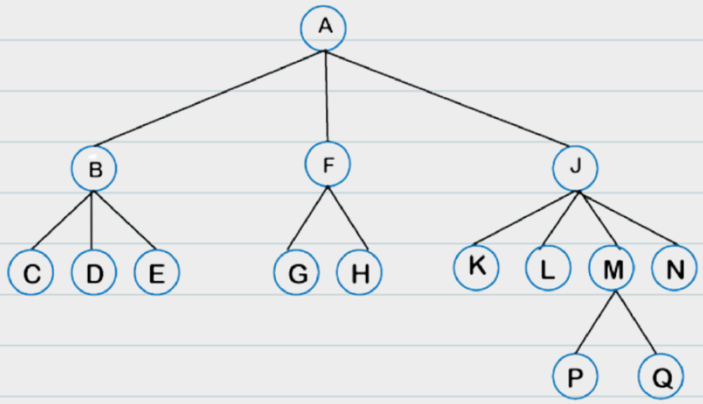
- **Descendants of a node**: children, grandchildren, grand-grandchildren etc. of a node.
- Descendants of F→ H, I, J, K, L, M
- **Depth of a node**: number of ancestors or length of the path from the root to the node. Depth of H→ 2
- **Height of a tree**: depth of the deepest node. → 3



Cont...

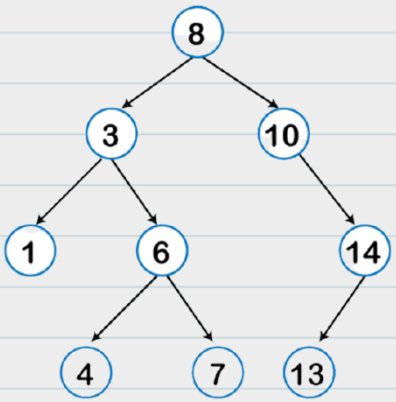
Types of Tree data structure

- **General tree** - In the general tree, a node can have either 0 or maximum n number of nodes. There is no restriction imposed on the degree of the node (the number of nodes that a node can contain).



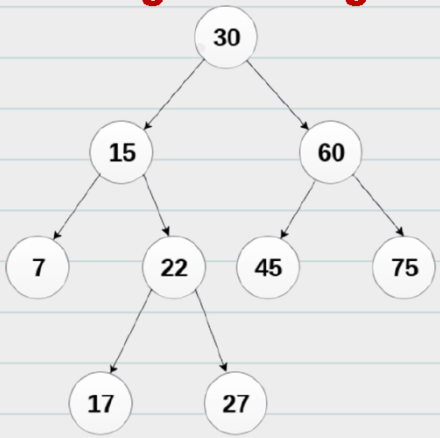
Cont...

- **Binary tree** - In a binary tree, each node in a tree can have utmost two child nodes. Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes.
- Binary Trees are used in the applications like expression evaluation and many more.



Cont...

- **Binary search tree** - is an ordered binary tree. All the elements in the left sub-tree are less than the root while elements present in the right sub-tree are greater than or equal to the root node element.
- They are used for searching, sorting, etc.



Cont...

Binary Search Tree (Ordered Binary Tree)

A binary tree that may be empty, but if it is not empty it satisfies the following.

- Every node has a key and no two elements have the same key.
- The keys in the right subtree are larger than the keys in the root.
- The keys in the left subtree are smaller than the keys in the root.
- The left & right subtrees are also binary search trees.

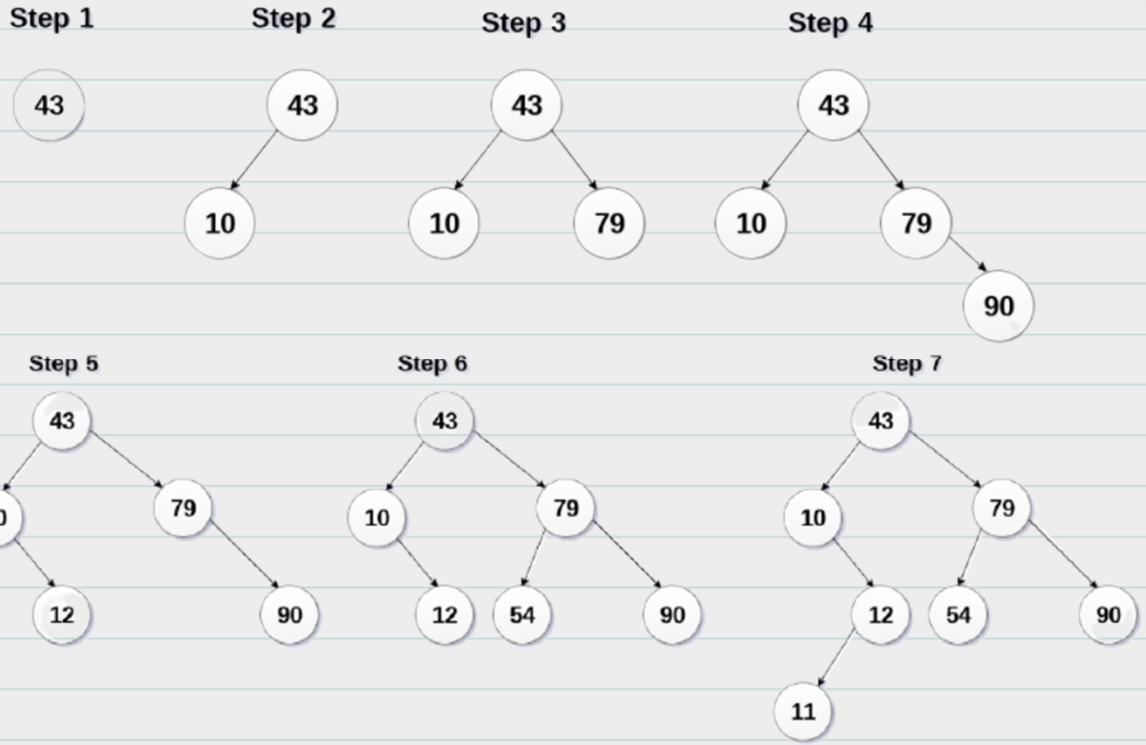
Cont...

Q1. Create the binary search tree using the following data elements. 43, 10, 79, 90, 12, 54, 11, 9, 50

1. Insert 43 into the tree as the root of the tree.
2. Read the next element, if it is lesser than the root node element, insert it as the root of the left sub-tree.
3. Otherwise, insert it as the root of the right of the right sub-tree.

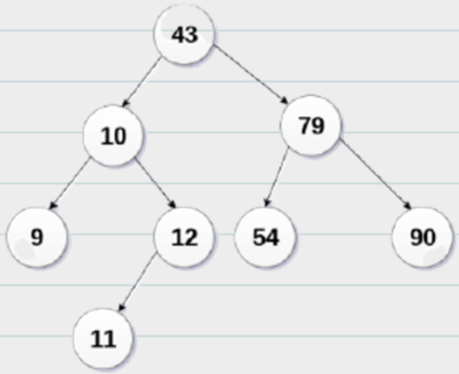
The process of creating BST by using the given elements, is shown in the image below.

Cont...

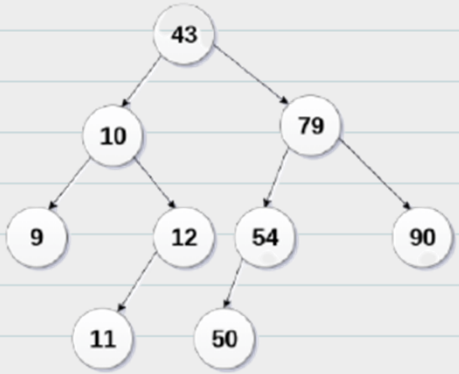

Step 1
43
Step 2
43
10
Step 3
43
10
79
Step 4
43
10
79
90
Step 5
43
10
79
12
90
Step 6
43
10
79
12
54
90
Step 7
43
10
79
12
54
90
11

Cont...

Step 8



Step 9



Cont...

Four types of operations on Binary Search Tree

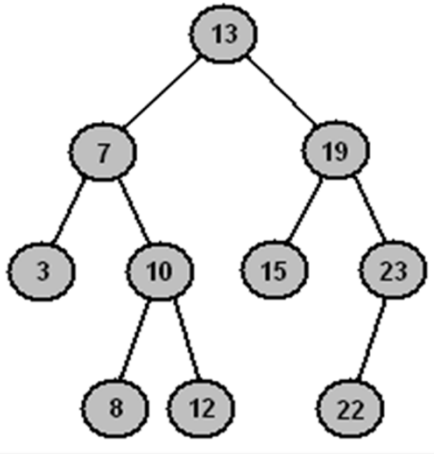
I. Tree Traversal

- **Tree traversal** is the process of visiting each node in the tree exactly one time.
- The order in which nodes are visited is not specified. Hence, there are many different ways in which tree traversal can be performed.
- Two of the most common and useful are known as **breadth-first** and **depth-first traversal**.

Cont...

a) *Breadth-first traversal*

- Breadth-first traversal starts with either the top or bottom level of the tree, and visits each node in the level (either from left-to-right or from right-to-left) before moving to the next level.
- Thus, there are four possibilities:
  - Top-to-bottom, left-to-right.
  - Top-to-bottom, right-to-left.
  - Bottom-to-top, left-to-right.
  - Bottom-to-top, right-to-left.



Cont...

- **For example**, a top-to-bottom, left-to-right breadth-first traversal of the tree in figure would result in the sequence.

  13, 7, 19, 3, 10, 15, 23, 8, 12, 22.

Cont...

b) Depth-first traversal

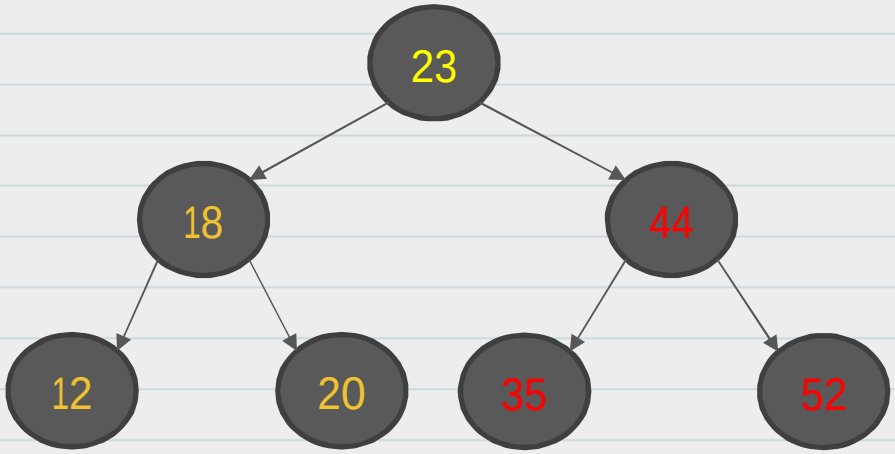
- **Breadth-first traversal** proceeds across each level before moving to the next, depth-first traversal moves as far possible down the left (or right) subtree, before backing up to the previous crossroads and taking the other subtree.
- This process is repeated until all nodes are visited.
- This definition, however, does not specify exactly when each node is visited: before visiting its child node(s) or after?

Cont...

At each node there are three tasks:

- **Pre order traversal** - traversing binary tree in the order of **Root**, **Left** and **Right**.
- **Inorder traversal** - traversing binary tree in the order of **Left**, **Root** and **Right**.
- **Postorder traversal** - traversing binary tree in the order of **Left**, **Right** and **Root**.

Cont...



Preorder traversal - 23,18,12,20,44,35,52

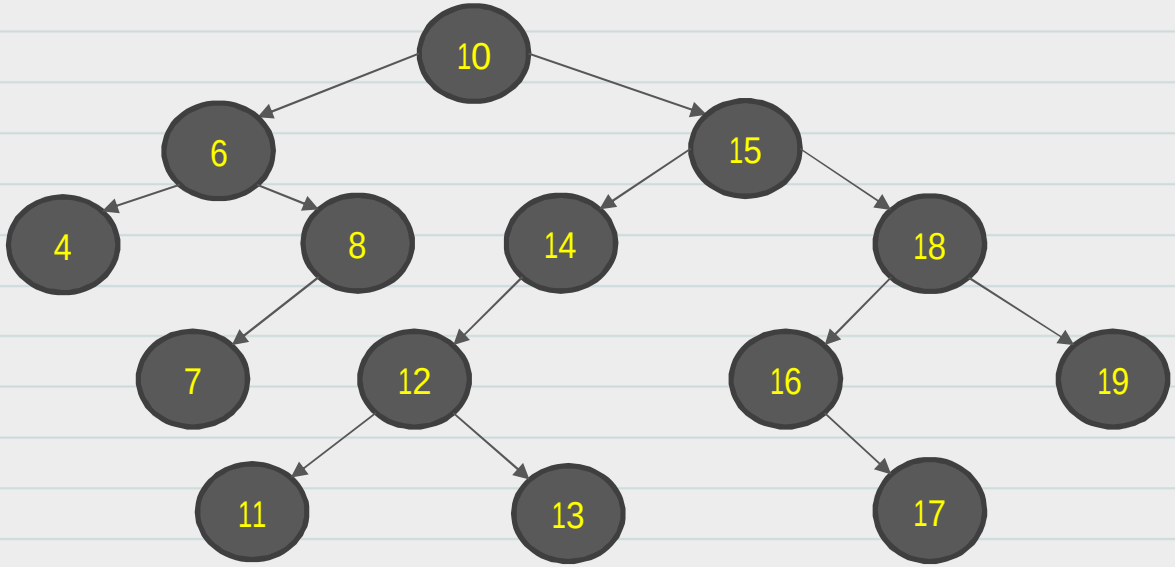
Inorder traversal - 12,18,20,23,35,44,52

Postorder traversal - 12,20,18,35,52,44,23

Cont...

Assignment 3

Find Preorder Traversal, Inorder Traversal and Postorder Traversal for the following tree?



Cont...

Four types of operations on Binary Search Tree

II. Searching in BST

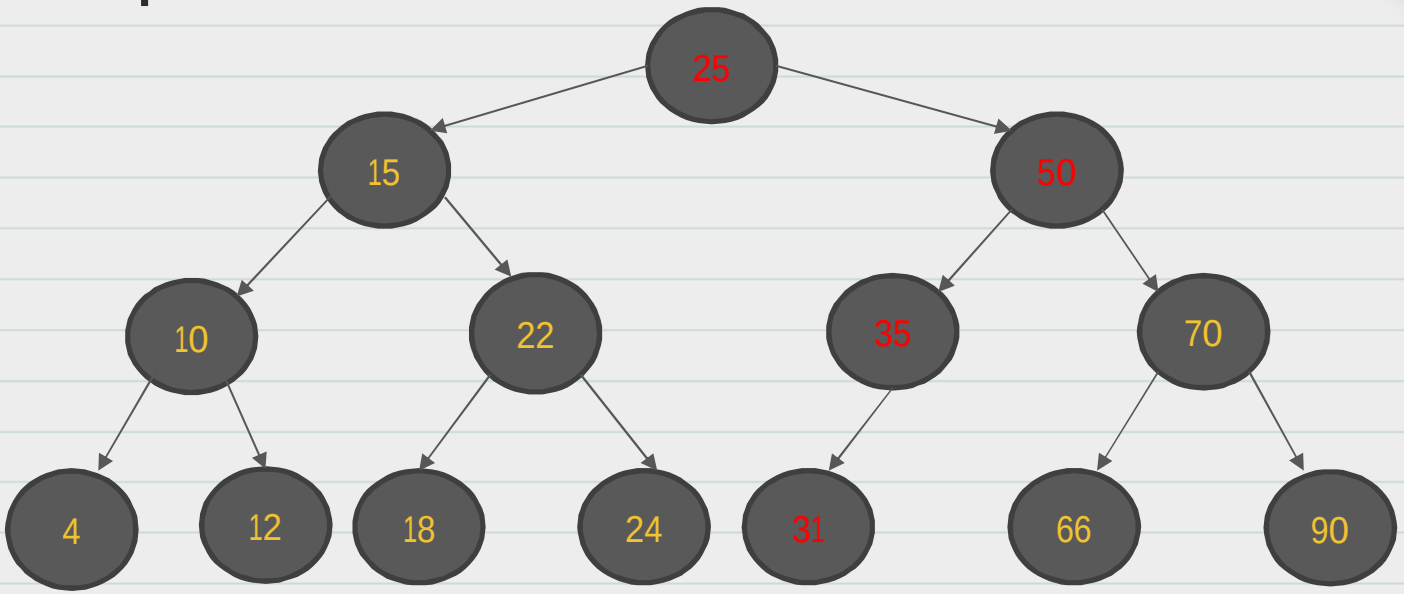
- Find a node in to the Binary Search Tree
- Start at the root nodes as current nodes.
- If the search key value matches the current nodes key then found a match.
- If the search key value is greater than current node
  1. If current node has right child, search in the right
  2. else no match node in the tree

Cont...

- If the search key value is less than current node
  1. If current node has left child, search in the left
  2. else no match node in the tree

Cont...

Example - Search 32 in the Tree



Cont...

- **Solution** - Start from the root node 25 and check 32>25 so focus on the right sub tree then check 32<50 then focus on the left side, check 32<35 focus on left sub tree found 31 after that there is no leaf child.
- So, Print No Match node in the tree

Cont...

Four types of operations on Binary Search Tree

III. Insertion in BST

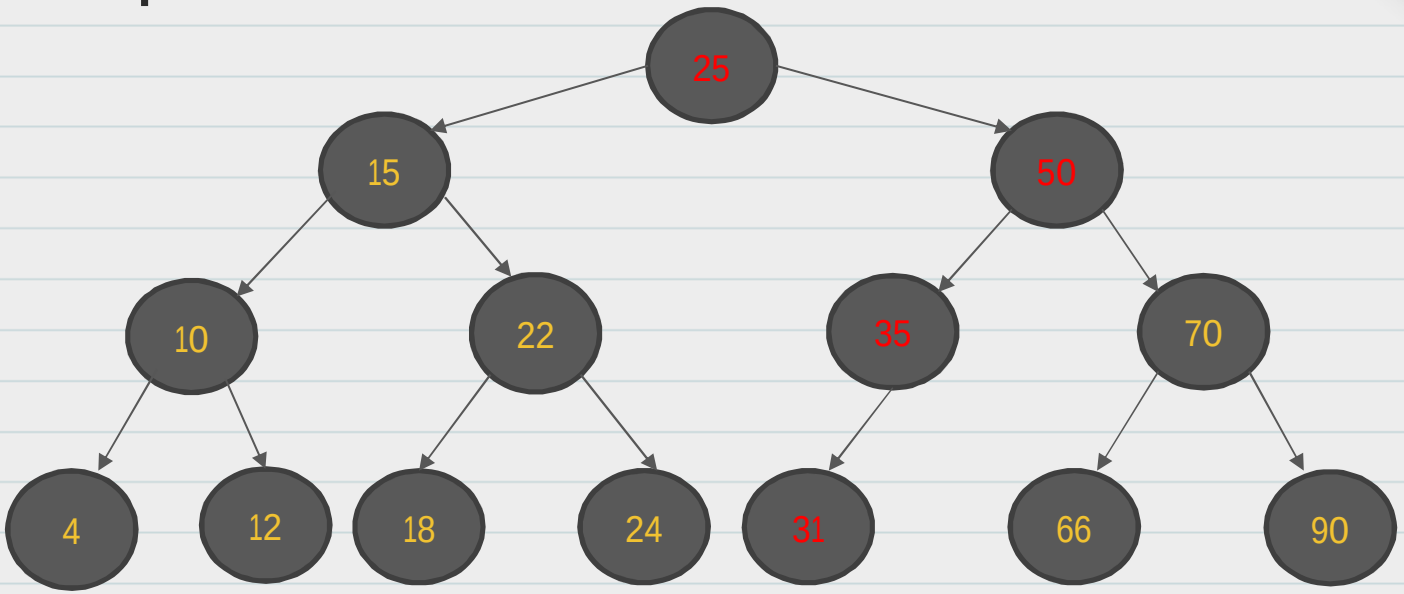
- The insertion process for binary search trees is actually very similar to the search process.
- Always insert new node as leaf node
- Start at root node as current node.
- If current new node key < current key
  1. If current node has left child, search in the left
  2. else add new node as current left child

Cont...

- If current new node key > current key
  1. If current node has Right child, search in the right
  2. else add new node as current right child

Cont...

Example - Insert 60 in the Tree



Cont...

- **Solution** - Start from the root node 25 and check 60>25 so focus on the right sub tree then check 60>50 then focus on the right side, check 60<70 focus on left sub tree found 66 after that there is no leaf at the left side of node 66 then insert 60.

Cont...

Four types of operations on Binary Search Tree

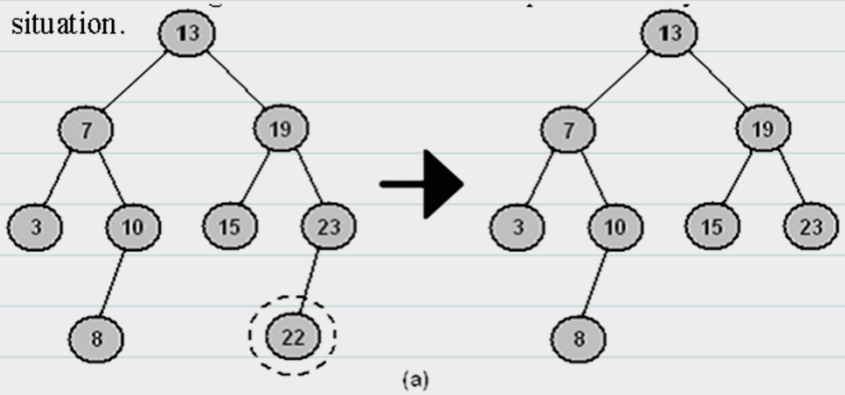
IV. Deletion

- Deletion of a node from a binary search tree can be more problematic.
- The difficulty of the operation depends on the position of the node in the tree.

There are three cases:

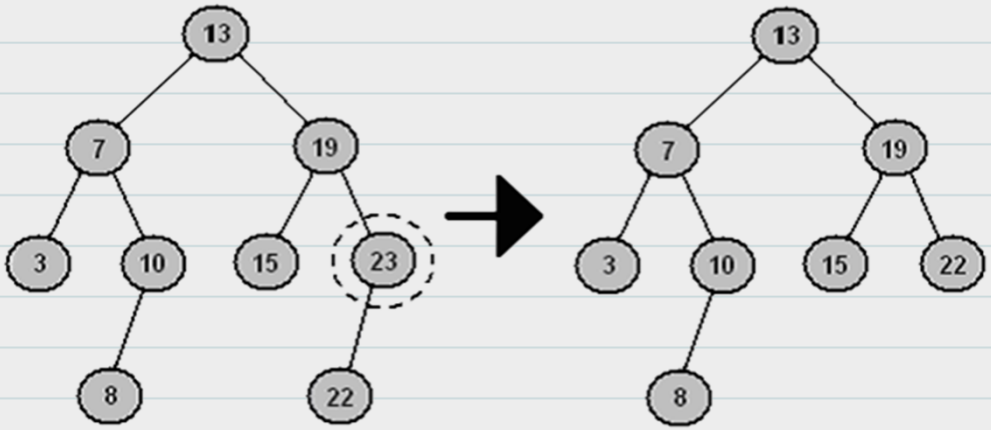
I. The **node is a leaf** - this is the easiest case to deal with. The appropriate pointer of its parent is set to null and the node deleted (see Figure a).

Cont...



II. The **node has one child**. This case is also not complicated. The appropriate pointer of its parent is modified to point to the child of the node (see Figure b).

Cont...

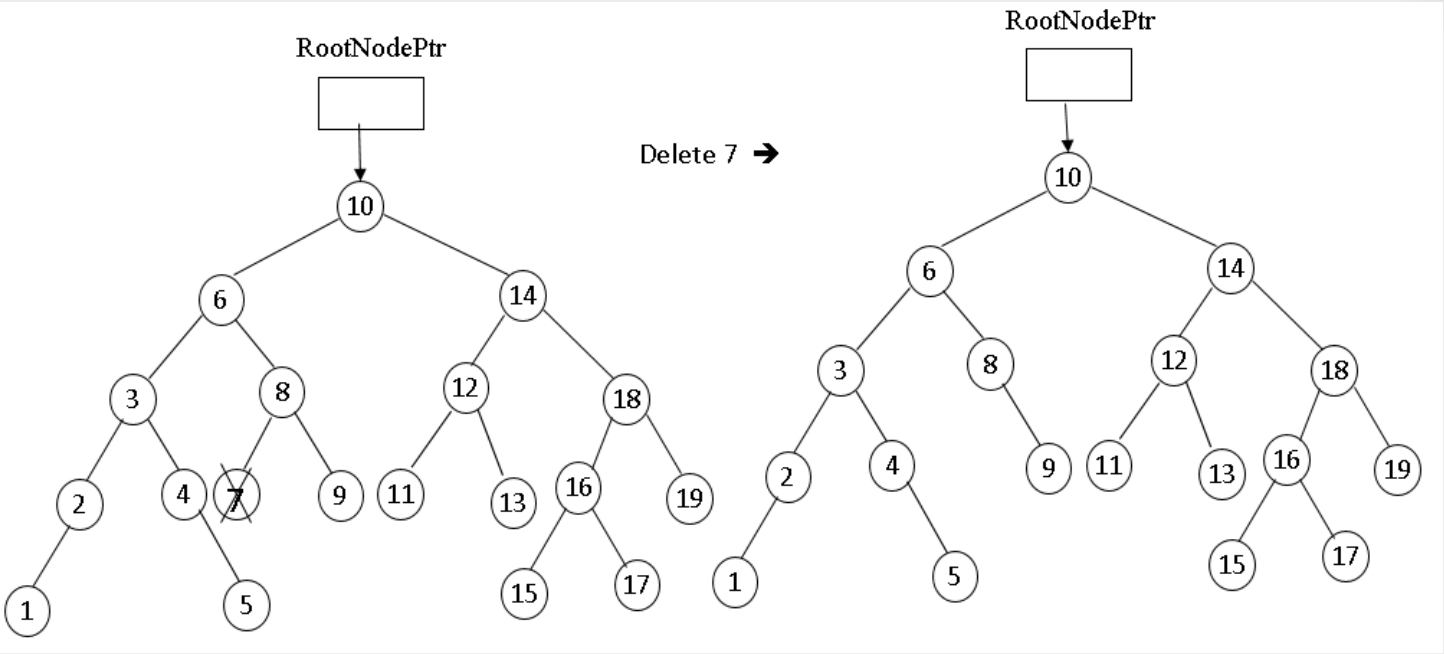


(b)

III. The **node has two children**. There is no simple way to delete nodes like this.

Cont...

Case 1- Deleting a leaf node (a node having no child), e.g. 7



Cont...

Case 2 - Deleting a node having only one child

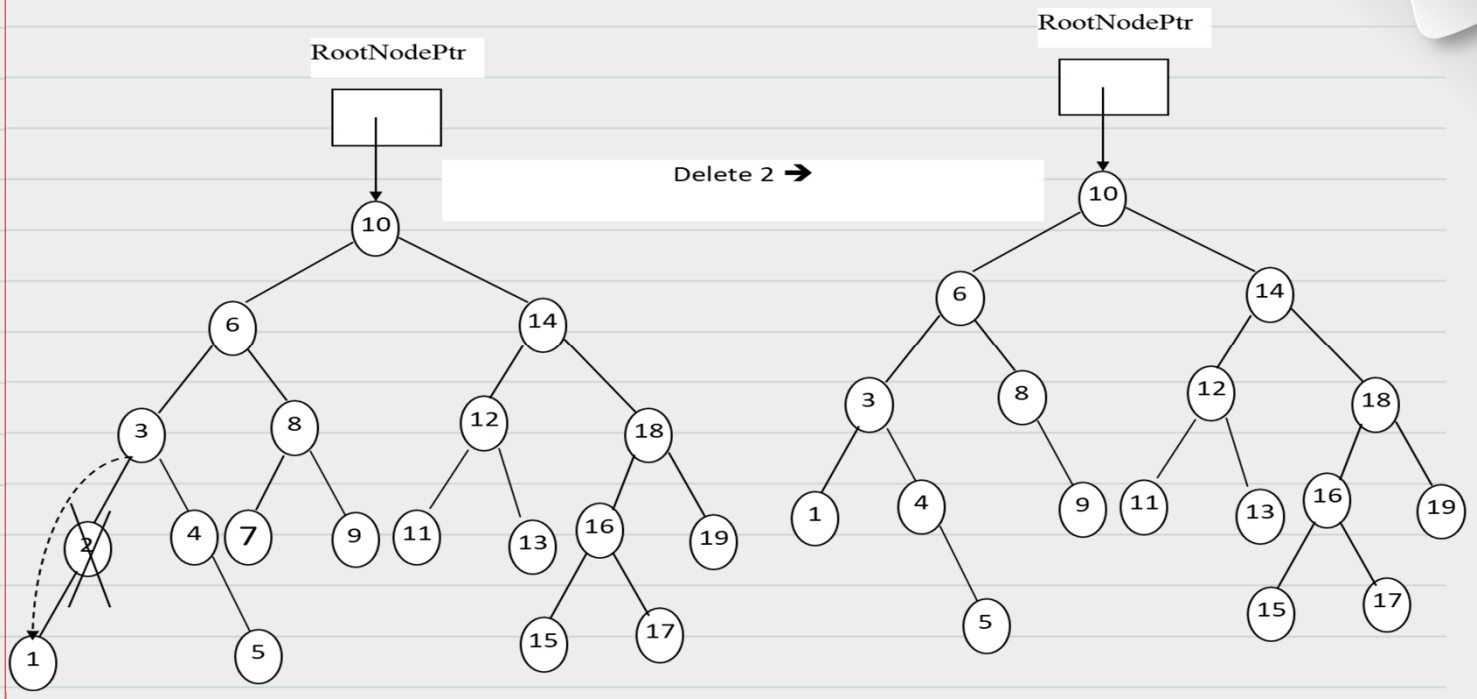
Approach 1: Deletion by merging – one of the following is done

- If the deleted node is the left child of its parent and the deleted node has only the left child, the left child of the deleted node is made the left child of the parent of the deleted node.
- If the deleted node is the left child of its parent and the deleted node has only the right child, the right child of the deleted node is made the left child of the parent of the deleted node.

Cont...

- If the deleted node is the right child of its parent and the node to be deleted has only the left child, the left child of the deleted node is made the right child of the parent of the deleted node.
- If the deleted node is the right child of its parent and the deleted node has only the right child, the right child of the deleted node is made the right child of the parent of the deleted node.

Cont...


RootNodePtr
Delete 2 ➔
10
6
14
3
8
12
18
2
4
7
9
11
13
16
19
1
5
15
17
RootNodePtr
10
6
14
3
8
12
18
1
4
9
11
13
16
19
5
15
17

Cont...

Case 3- Deleting the root node, 10

Approach 1: Deletion by merging- one of the following is done

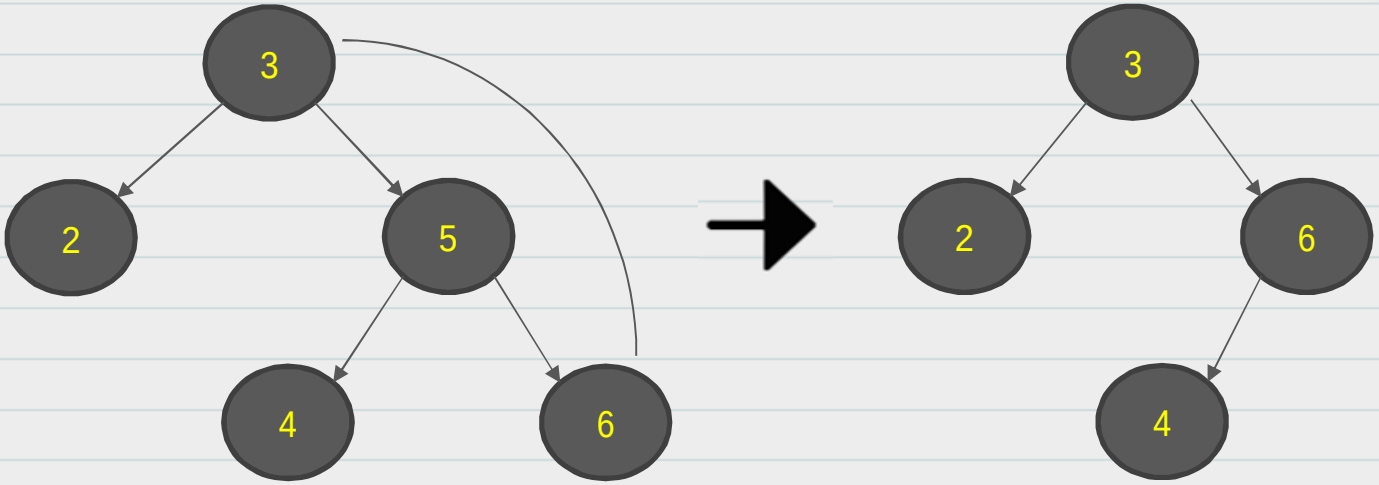
- If the tree has only one node the root node pointer is made to point to nothing (NULL)
- **If the root node has left child**
  - the root node pointer is made to point to left child
  - the right child of the root node is made the right child of the node containing the largest element in the left of the root node

Cont...

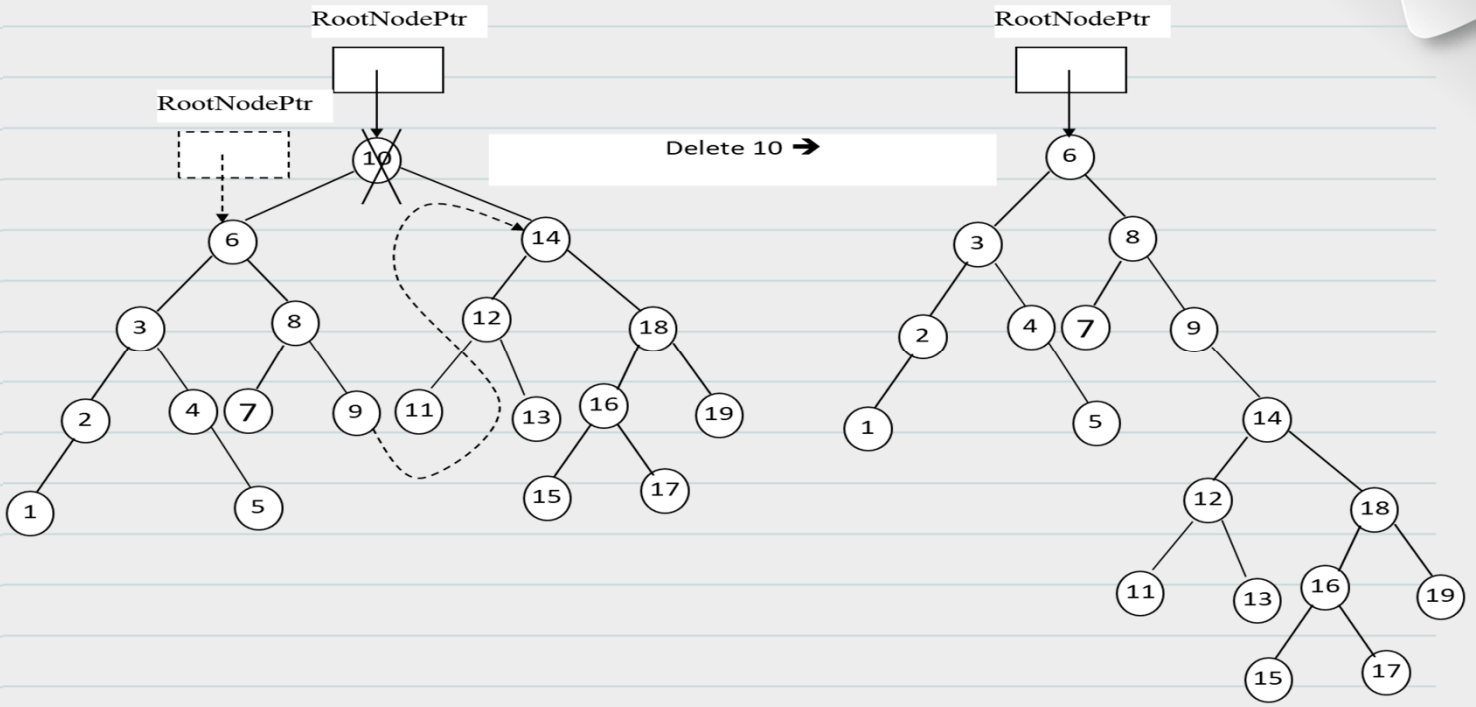
- **If root node has right child**
  - the root node pointer is made to point to the right child
  - the left child of the root node is made the left child of the node containing the smallest element in the right of the root node

Cont...

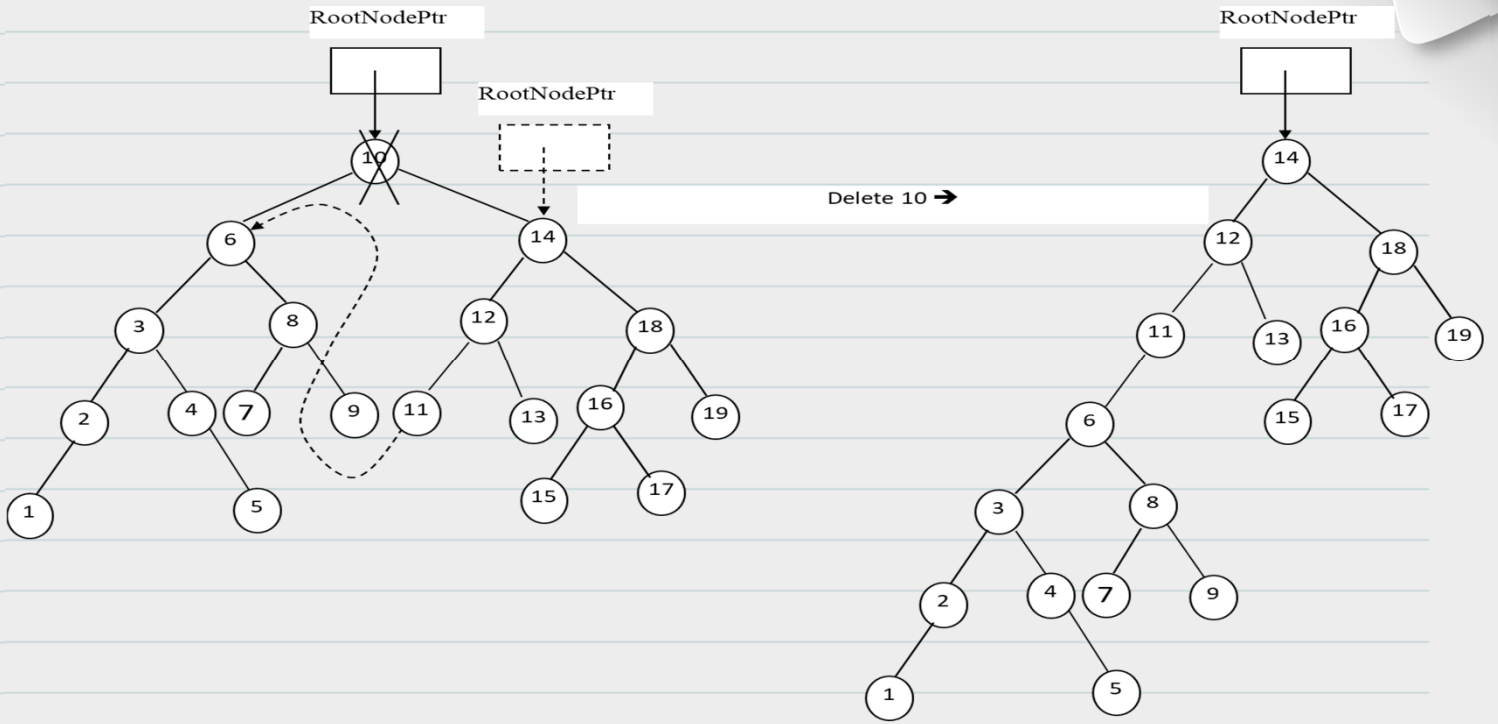
- A node Having Two Child



Cont...


RootNodePtr
RootNodePtr
10
6
14
3
8
12
18
2
4
7
9
11
13
16
19
1
5
15
17
Delete 10 ➔
RootNodePtr
6
3
8
2
4
7
9
1
5
14
12
18
11
13
16
19
15
17

Cont...


RootNodePtr
RootNodePtr
10
6
14
3
8
12
18
2
4
7
9
11
13
16
19
1
5
15
17
Delete 10
RootNodePtr
14
12
18
11
13
16
19
6
15
17
3
8
2
4
7
9
1
5

4.2 - Graphs

- **Graphs** can be defined as the pictorial representation of the set of elements (represented by vertices) connected by the links known as edges.
- A **graph** is a data structure in which any node can link to any other node.
- A graph is different from tree in the sense that a graph can have cycle while the tree can not have the one.
- A simple graph G=(V, E) consists of a nonempty set of vertices V and a (possibly empty) set of edges E.
- Figure 10a-c shows examples of simple graphs.

Cont...

- Note that there must be at least one vertex but there need not be any edges, as in Figure 10a.
- Also note that a graph may consist of two unconnected sets of nodes, as in Figure 10b.

Cont...

Graph Terminology

- A **complete graph** is one in which every node is connected to every other node (Figure 10d).
- A **multigraph** is a graph in which two nodes can be joined by multiple edges (Figure10e).
- A **pseudograph** is a graph in which loops can occur, i.e. a node can be joined to itself (Figure 10f).
- A **directed graph**, or **digraph**, is one in which the edges have a direction (Figure10g).
- Finally, a **weighted graph** is one in which each edge has an associated value, or weight (Figure10h).

Cont...

- **Examples of Graphs**

(a-c) simple graphs;

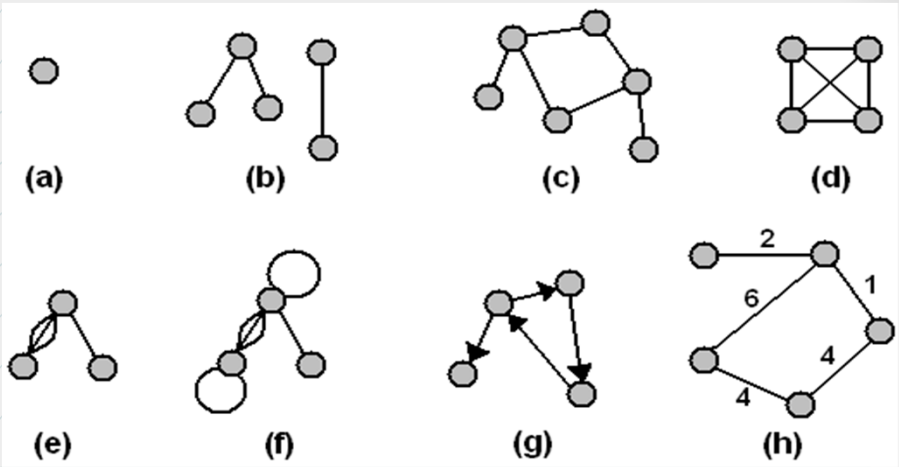
(d) a complete graph;

(e) a multigraph;

(f) a pseudograph;

(g) a digraph;

(h) a weighted graph



Cont...

Graph Representation

- Simply mean the technique to be used to store some graph into the computer memory.
- Graphs can be represented using either static or dynamic data structures.
- One common representation is to use a two-dimensional array called an adjacency matrix.
- An adjacency matrix of a graph G=(V, E) is a |V| by |V| array where each element aij of the array is defined as

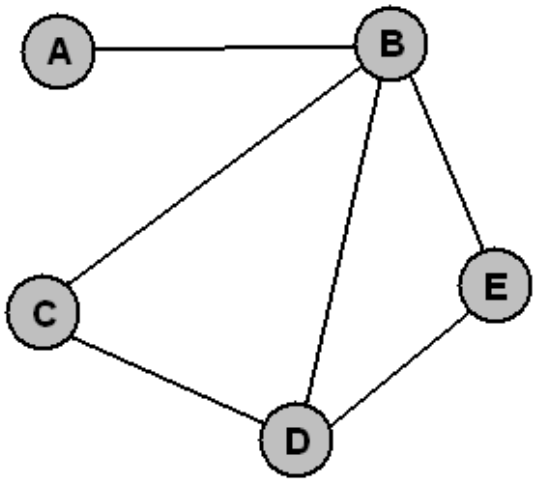
$$a_{ij} = \begin{cases} 1 & \text{if there exists an edge between } v_i \text{ and } v_j \\ 0 & \text{otherwise} \end{cases}$$

Cont...

- Finally, graphs can also be implemented using dynamic data structures.
- Each GraphVertex consists of a name and a linked list of edges.
- Each EdgeNode contains a pointer to a GraphNode (i.e. an edge) and a link to the next edge emanating from this vertex.
- Therefore there is no limit to how many edges a vertex can have.

Cont...

Graph Representations Using Matrices



Adjacency Matrix

	A	B	C	D	E
A	0	1	0	0	0
B	1	0	1	1	1
C	0	1	0	1	0
D	0	1	1	0	1
E	0	1	0	1	0

Incidence Matrix

	AB	BC	BD	BE	CD	DE
A	1	0	0	0	0	0
B	1	1	1	1	0	0
C	0	1	0	0	1	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1


END
Thank You!